Informe del desarrollo y aseguramiento de calidad de UniFood

Fecha de revisión técnica: 28 de octubre de 2026. Integrantes: Ian Romero, Kathleen Barrientos y Sergio Iporre.

Introducción

UniFood es una aplicación web para gestionar pedidos de alimentos dentro del entorno universitario. Permite consultar el catálogo, preparar un carrito, confirmar pedidos y organizar su preparación y entrega. El trabajo incorpora aseguramiento de calidad de software mediante requisitos verificables, un Test Plan, casos de prueba, revisión del código, automatización y registro de resultados.

El desarrollo reúne el flujo principal del estudiante, las operaciones del personal y la administración del catálogo. La revisión más reciente fortaleció la confirmación de pedidos, las transiciones de estado, la generación de códigos y el cálculo de importes. La verificación local del 28 de octubre registró 58 pruebas PHP aprobadas con 286 aserciones. También se verificaron seis pruebas del service worker de la PWA.

Este informe explica las funcionalidades implementadas, las correcciones realizadas y la evidencia que permite comprobar su comportamiento. Los resultados de la campaña local se presentan separados de la campaña de navegador previamente aprobada en GitHub Actions, para mantener la relación entre cada ejecución y la versión evaluada.

Objetivo y alcance del sistema

El objetivo de UniFood es facilitar la solicitud y atención de pedidos en la tienda universitaria. El estudiante selecciona productos y paga al recoger; el personal administra la preparación y la entrega, mientras el administrador mantiene el catálogo.

La aplicación diferencia tres roles de acceso. El estudiante utiliza catálogo, carrito y consulta de pedidos propios. El empleado atiende los pedidos desde el panel del personal. El administrador dispone de las funciones del personal y del mantenimiento de productos. La autorización se comprueba en el servidor mediante rutas protegidas y middleware de roles.

El alcance desarrollado comprende autenticación, registro, recuperación de contraseña para demostración, catálogo, carrito, pedidos, cancelación, avance de estados y administración de productos. La interfaz se adapta a pantallas móviles e incorpora una PWA con manifiesto, iconos y pantalla genérica sin conexión. Las operaciones del negocio se realizan con conexión al servidor y el pago es presencial.

Funcionalidades implementadas

Autenticación y control de acceso

Se implementaron registro de estudiantes, inicio y cierre de sesión, redirección según el rol y protección de las operaciones. Al iniciar sesión se regenera el identificador de sesión. El cierre invalida la sesión y renueva el token de protección del formulario.

La recuperación de contraseña utiliza el mecanismo de Laravel para almacenar el hash del token, comprobar su vigencia y evitar su reutilización. En el entorno de demostración, el enlace se obtiene mediante el registro del servidor. Las pruebas verifican el cambio de contraseña, el rechazo de un token vencido y la limitación de solicitudes repetidas.

Catálogo y administración de productos

El catálogo muestra productos activos, sus precios y su disponibilidad. Las propiedades Activo y Disponible tienen significados distintos: un producto activo puede mostrarse como agotado, mientras que uno inactivo se retira del catálogo del estudiante.

El administrador puede crear y editar productos, cambiar disponibilidad y desactivar registros mediante baja lógica. Se conservan los datos necesarios para relacionar los productos con pedidos anteriores.

También se corrigió el tratamiento de las casillas desmarcadas para que su valor se guarde como falso y se respete al consultar el catálogo.

Carrito y creación de pedidos

El carrito permite agregar, actualizar y retirar productos. El servidor consulta los precios de la base de datos y calcula los subtotales y el total. Antes de registrar un pedido vuelve a comprobar que los productos existan, estén activos y se encuentren disponibles.

La confirmación genera un pedido pendiente y sus detalles dentro de una transacción. Los detalles guardan una copia del nombre y del precio unitario, junto con cantidad y subtotal. Esto conserva la información de la compra aunque posteriormente se modifique el catálogo. Tras confirmar correctamente, se vacía el carrito.

Consulta y atención de pedidos

Cada estudiante puede consultar únicamente sus pedidos. El intento de acceder al detalle de un pedido de otra cuenta recibe un rechazo HTTP 403. El personal puede consultar los pedidos y avanzar por la secuencia pendiente, en preparación, listo para recoger y entregado.

La cancelación del estudiante solo se aplica mientras el pedido continúa pendiente. Los pedidos entregados se mantienen sin modificaciones; un intento de avanzar nuevamente un pedido entregado recibe HTTP 409. Las vistas muestran el estado y se actualizan automáticamente para facilitar su seguimiento.

Aseguramiento de calidad aplicado

El Test Plan TP UF 01 organiza introducción, alcance, estrategia, responsabilidades, ciclo de pruebas, criterios de entrada y salida, métricas y riesgos. Los requisitos funcionales se relacionan con historias de usuario y casos de prueba mediante identificadores RF, HU y CP.

La trazabilidad permite explicar qué se comprueba y con qué evidencia. Por ejemplo, RF 05 exige que el estudiante consulte exclusivamente sus pedidos; HU 05 expresa esa necesidad; y CP 08 verifica el rechazo del acceso al pedido de otro estudiante. La prueba automatizada comprueba tanto la respuesta del servidor como la protección de la información.

Se distinguieron actividades de aseguramiento y de control. El aseguramiento de calidad incluye revisar requisitos, establecer reglas y organizar las verificaciones. El control de calidad comprende ejecutar casos y comparar los resultados obtenidos con los esperados. La revisión estática del código se complementó con pruebas funcionales, de integración, regresión y componentes.

Las técnicas aplicadas incluyen particiones de equivalencia para entradas válidas e inválidas, valores límite para cantidades y precios, tablas de decisión para permisos y disponibilidad, y transiciones de estado para los pedidos. La selección de pruebas priorizó operaciones que afectan compras, importes, acceso y consistencia de los datos.

Organización del trabajo con Scrum

Se documentaron historias de usuario con criterios de aceptación, un Product Backlog, un tablero en GitHub Projects y criterios de preparación y terminación. Los enlaces entre historias, casos, cambios y evidencias permiten seguir el trabajo desde la necesidad del usuario hasta su comprobación técnica.

La distribución acordada asigna a Ian Romero la responsabilidad de Product Owner, a Kathleen Barrientos la de Scrum Master y a Sergio Iporre un foco en desarrollo y automatización. El desarrollo y la calidad son responsabilidades compartidas por los tres integrantes. Estos roles del equipo son diferentes de los roles estudiante, empleado y administrador de la aplicación.

El Product Owner orienta prioridades y criterios de aceptación; la Scrum Master facilita el seguimiento y la coordinación; y el trabajo de desarrollo incluye implementar, probar y corregir. La documentación

define cómo relacionar estas responsabilidades con la planificación, revisión del incremento y mejora del proceso.

Mejoras técnicas realizadas

Confirmación de pedidos con efecto único

Se reforzó la protección contra confirmaciones duplicadas. El servidor exige un identificador UUID de confirmación y comprueba su correspondencia con la sesión. El identificador se almacena en el pedido mediante una columna con restricción única, de modo que una misma confirmación no pueda registrarse dos veces.

La protección combina el bloqueo temporal por usuario, el registro persistente del identificador y el tratamiento de reintentos. Una repetición se reconoce como ya procesada. Si llega una sesión antigua, se retira su carrito y su identificador consumido; si la sesión ya contiene un carrito más reciente, ese nuevo carrito se conserva. En la interfaz, el botón se desactiva mientras se envía la solicitud.

Esta propiedad se denomina idempotencia: repetir la misma solicitud conserva un solo efecto. Se verificó mediante regresiones de reenvío y mediante dos procesos PHP separados que confirmaron el mismo identificador sobre una base SQLite compartida.

Actualización condicional de estados

El formulario del personal incorpora el estado que mostraba la pantalla. El servidor modifica el pedido únicamente si su estado guardado coincide con ese valor. La comprobación y la escritura forman parte de una misma actualización condicional en la base de datos.

Así, repetir una acción desde una pantalla antigua no avanza dos pasos. La cancelación también se limita mediante una condición que exige el estado pendiente. Cuando cancelar e iniciar preparación compiten, solo una de las dos operaciones puede aplicarse al pedido.

Para SQLite se configuraron transacciones IMMEDIATE y una espera de cinco segundos por bloqueos de escritura. Esta configuración evita depender de un bloqueo de filas que SQLite no genera con lockForUpdate y permite ordenar las escrituras de las transacciones utilizadas en el flujo de compra.

Numeración diaria persistente

La generación del código de pedido se trasladó desde un contador en caché a un contador diario en la base de datos. Se mantiene el formato UF seguido de la fecha y el número correlativo. La reserva se realiza dentro de una transacción, y la columna de código conserva su restricción de unicidad.

Al inicializar el contador de un día se consideran los códigos existentes. La numeración continúa aunque se limpie la caché. Las pruebas comprobaron reservas consecutivas, recuperación a partir de pedidos históricos y asignación de códigos distintos desde dos procesos separados.

Cálculo exacto de importes

Se incorporó una clase de importes que convierte precios decimales en centavos enteros. Los subtotales y el total se calculan con enteros y luego se convierten nuevamente a una representación decimal de dos posiciones para su almacenamiento y presentación.

Por ejemplo, un precio de Bs 0,10 por tres unidades se calcula como 10 por 3, cuyo resultado es 30 centavos, equivalente a Bs 0,30. La regresión inicial mostró que el cálculo anterior con float producía internamente 0.30000000000000004. La corrección evita esa aproximación en la aritmética de la compra.

Validación de cantidades y precios

El servidor comprueba un máximo de 50 unidades por producto, tanto por solicitud como por acumulación de agregados. Los precios admiten hasta dos decimales y un máximo coherente con la columna del

producto. También se rechazan pedidos cuyo total supera la capacidad decimal definida para su almacenamiento.

Estas validaciones refuerzan las restricciones visibles del formulario. Además, se corrigió el registro y la edición de productos cuando se omite la descripción opcional, evitando un error por tratar ese campo como obligatorio después de la validación.

Limitación de intentos de acceso

El inicio de sesión cuenta los intentos fallidos por combinación de correo e IP. Después de cinco fallos, el siguiente intento se bloquea durante la ventana configurada de 60 segundos. La interfaz muestra el aviso y el tiempo de espera; el acceso correcto limpia el contador de fallos.

Las regresiones comprueban el bloqueo, la limpieza tras un acceso correcto y la posibilidad de volver a iniciar sesión después de que vence la espera.

Pruebas y resultados comprobados

Se conservó un reporte inicial con siete regresiones fallidas antes de corregir confirmaciones, estados e importes. Después se ejecutaron las pruebas sobre el comportamiento corregido. Otra regresión reprodujo la repetición de un código reservado al limpiar la caché. Este procedimiento aporta evidencia concreta del ciclo prueba que falla, corrección y nueva ejecución aprobada.

La suite incluye pruebas funcionales e integración con Laravel, pruebas unitarias de importes y pruebas de concurrencia mediante procesos PHP independientes. Las pruebas habituales utilizan SQLite en memoria; los escenarios concurrentes emplean archivos temporales independientes de la base de demostración.

Campaña	Herramienta	Resultado
Local del 28 de octubre de 2026	PHPUnit	58 de 58 pruebas aprobadas y 286 aserciones
Local de esta actualización	Node test runner del service worker	6 de 6 pruebas aprobadas
Campaña CI 40628543184	PHPUnit	14 de 14 pruebas aprobadas y 110 aserciones
Campaña CI 40628543184	PWA	6 de 6 pruebas aprobadas
Campaña CI 40628543184	Cypress	7 de 7 pruebas aprobadas
Campaña CI 40628543184	Selenium	4 de 4 pruebas aprobadas

La campaña histórica de GitHub Actions corresponde al código 14774aa01fc7d735df3a5dc9e6b65cdff87d1f32. La campaña local corresponde a los cambios de esta revisión sobre la base e5631aa. Cada resultado se atribuye a su ejecución; las cifras de ambas campañas no se suman como una sola medición.

Las comprobaciones concurrentes abarcan cuatro conflictos: dos confirmaciones con el mismo identificador, dos reservas de código, dos avances desde el mismo estado y cancelación frente a preparación. Una prueba adicional de migración verifica la conservación de pedidos históricos y la continuación de su numeración.

La tasa de aprobación local del backend es 58 dividido entre 58 por 100, igual a 100 por ciento de las pruebas ejecutadas. Esta métrica expresa el resultado de la suite y se mantiene diferenciada del concepto de cobertura de código. El aumento desde las 28 pruebas observadas al inicio de la revisión hasta las 58 actuales representa 30 verificaciones adicionales.

Automatización y evidencias

El repositorio incorpora un workflow de GitHub Actions que instala dependencias, compila la interfaz, prepara una base aislada, ejecuta pruebas PHP, PWA, Cypress y Selenium, y conserva reportes y capturas como artefactos. Se verificó que la campaña histórica citada terminó correctamente.

Los reportes de esta revisión se guardaron en evidencias_sqa/automatizacion/2026-10-28. El archivo phpunit.xml registra la ejecución local del backend, pwa.xml conserva las pruebas del service worker y regresion-antes.xml documenta los siete casos iniciales fallidos. La compilación de la interfaz con Vite también se ejecutó correctamente.

La automatización facilita repetir las comprobaciones después de un cambio y relacionar una corrección con su regresión. Los permisos, la privacidad de pedidos, las casillas de productos y los controles de contraseña cuentan además con pruebas específicas en el repositorio.

Conclusiones

UniFood dispone del flujo principal de pedidos universitarios y de controles de acceso para sus tres actores. El trabajo de SQA organizó requisitos, historias, reglas de negocio, Test Plan y casos de prueba, y conectó su comprobación con herramientas de automatización y seguimiento.

Las mejoras realizadas fortalecieron operaciones sensibles del sistema: una confirmación conserva un solo efecto, los cambios de estado se aplican de forma condicional, los códigos utilizan numeración persistente y los importes se calculan en centavos. Las validaciones del servidor y las regresiones de acceso complementan estas correcciones.

Los resultados locales de 58 pruebas PHP y seis pruebas de la PWA, junto con la campaña histórica aprobada en GitHub Actions, proporcionan evidencia identificable de las verificaciones ejecutadas. La relación entre defecto, prueba, corrección y resultado permite explicar y defender técnicamente el trabajo realizado.

Referencias y registros

1. Universidad del Valle. Plan SQA web. Actividad grupal Actualización del Test Plan en un entorno ágil. Material de la asignatura.

2. UniFood. Plan de aseguramiento y pruebas, versión 1.3, 27 de octubre de 2026. Documento y anexos conservados en docs/sqa.

3. UniFood. Código fuente, migraciones y suites de prueba revisadas el 28 de octubre de 2026.

4. Repositorio del proyecto: https://github.com/ifrancro/UniFood.

5. Campaña automatizada aprobada: https://github.com/ifrancro/UniFood/actions/runs/40628543184.

6. Reportes locales: evidencias_sqa/automatizacion/2026-10-28/phpunit.xml, pwa.xml y regresion-antes.xml.